

MOVELINK

System-Dokumentation & Traceability-Report

Anforderungs- & Abdeckungsanalyse der MoveLink App

Status: Freigegeben

Sprache: Deutsch

Autoren: Erlind & AI Assistant Antigravity

Dokumentenversion: v1.1.0 (Scraped Live)

Veröffentlichungsdatum: 20. Juni 2026

Gliederung (Inhaltsverzeichnis)

Dieses Dokument gliedert sich in die folgenden Abschnitte:

1. Anwendungsfälle (Use Cases)	3
2. Systemanforderungen (Requirements)	5
Funktionale Anforderungen //Was für Instanzen existieren denn eigentlich.	5
Nicht-funktionale Anforderungen	5
Randbedingungen	5
3. Architekturentscheidungen (ADR / Abwägungen)	7
Abwägungen	7
4. System-Architektur (C4 Modell)	9
System-Kontext-Diagramm (C4 Level 1)	9
System-Daten- & Kontrollfluss-Diagramm	9
4.1 Embedded Sensor-Firmware (Xiao MCU)	10
Architektur-Entscheidung	12
4.2 Mobile App (React Native)	13
C4-Architektur-Ebene	13
Beschreibung	13
Requirements	14
Datenfluss	14
Komponenten in diesem Container	14
5. Rückverfolgbarkeits-Matrix (Traceability)	16

1. Anwendungsfälle (Use Cases)

In diesem Abschnitt werden die primären Anwendungsfälle (Use Cases) des MoveLink-Systems beschrieben. Diese Use Cases definieren die Benutzerinteraktionen und Systemantworten für das Verbinden von Geräten, das Echtzeit-Training und das Einsehen historischer Daten.

Hier werden die primären Interaktionen zwischen dem Trainierenden und dem MoveLink-System beschrieben.

UC-1 – Trainingsgerät verbinden

- **Akteur:** Trainierender
- **Vorbedingung:** Trainingsgerät ist eingeschaltet und befindet sich in Reichweite.
- **Beschreibung:** Als Trainierender möchte ich mein Trainingsgerät mit der App verbinden, um Trainingsdaten erfassen zu können.
- **Ablauf (Szenario):**
 1. **Eingabe:** Der Trainierende öffnet die App.
- **Ausgabe*:** Die App zeigt eine Möglichkeit/einen Reiter für Hardwaregeräte an.
- 2. **Eingabe:** Der Trainierende klicke auf den Reiter "Hardwaregeräte".
- **Ausgabe*:** Die App zeigt eine Liste der verfügbaren Hardwaregeräte sowie den aktuellen Verbindungsstatus an.
- 3. **Eingabe:** Der Trainierende wählt ein Hardwaregerät aus der Liste aus und klickt auf "Verbinden".
- **Ausgabe*:** Die App zeigt die Detailansicht des ausgewählten Geräts und bestätigt den erfolgreichen Verbindungsaufbau.

UC-2 – Echtzeit-Training überwachen

- **Akteur:** Trainierender
- **Vorbedingung:** Das Trainingsgerät ist erfolgreich mit der App verbunden.
- **Beschreibung:** Ich als Trainierender möchte mein Training in Echtzeit überwachen können, um direkt Feedback zu meiner Ausführung zu erhalten.
- **Ablauf (Szenario):**
 1. **Eingabe:** Der Trainierende öffnet die App.
- **Ausgabe*:** Die App zeigt die Option zum Starten eines Trainings an.
- 2. **Eingabe:** Der Trainierende klickt auf "Training starten".
- **Ausgabe*:** Die App zeigt die Detailansicht des ausgewählten Trainings sowie die Start-Schaltfläche.
- 3. **Eingabe:** Der Trainierende drückt den "Start"-Button.
- **Ausgabe:** Die App demonstriert grafisch die auszuführende Übungsbewegung. (Bezug: **FA8**)*

4. **Eingabe:** Der Trainierende führt die Bewegung aus.

- **Ausgabe:** Die App visualisiert die Bewegung in Echtzeit, vergleicht sie mit der Zielvorgabe und gibt positives Feedback bei korrekter Ausführung. (Bezug: FA5, FA6, FA9)*

UC-3 – Trainingsdaten einsehen

- **Akteur:** Trainierender
- **Vorbedingung:** Mindestens eine aufgezeichnete Trainingseinheit ist in der Datenbank vorhanden.
- **Beschreibung:** Ich als Trainierender möchte vergangene Trainingseinheiten einsehen können, um meine Fortschritte zu verfolgen.
- **Ablauf (Szenario):**
 1. **Eingabe:** Der Trainierende öffnet die App.
 - **Ausgabe*:** Die App bietet eine Option zum Einsehen des Trainingsverlaufs.
 2. **Eingabe:** Der Trainierende navigiert zum Reiter "Trainingseinheiten".
 - **Ausgabe*:** Die App listet alle vergangenen Trainingseinheiten chronologisch auf.
 3. **Eingabe:** Der Trainierende wählt eine Trainingseinheit aus der Liste aus.
 - **Ausgabe*:** Die App bereitet die historischen Bewegungsdaten grafisch und statistisch auf.

2. Systemanforderungen (Requirements)

Dieser Abschnitt enthält die funktionalen Anforderungen (FA), nicht-funktionalen Anforderungen (NF) und Randbedingungen (R) des MoveLink-Systems.

Dieses Dokument definiert die funktionalen und nicht-funktionalen Anforderungen sowie die Randbedingungen des MoveLink-Systems.

Funktionale Anforderungen //Was für Instanzen existieren denn eigentlich.

FA1 – Es wird eine Applikation angeboten, welche als Input für die Steuerung für den Benutzer dient. (UC-1, UC-2, UC-3)

FA2 – Zu dem System wird ein Trainingsgerät benötigt, welche als Input für der Bewegungen des Benutzers dient. Dieses Gerät gibt die Sensorwerte als auch die Art der Bewegung zurück. (UC-1, UC-2)

FA3 – Es wird ein Ort benötigt, welche die Daten des Benutzers speichert. (UC-1, UC-3)

Nicht-funktionale Anforderungen

NF1 – Latenz

Die End-to-End-Latenz von der physischen Sensorbewegung bis zur visuellen Darstellung in der App muss ≤ 100 ms sein.

NF2 – Zuverlässigkeit und Reconnect

Bei Verbindungsabbrüchen muss die App den Nutzer umgehend benachrichtigen und automatische Wiederverbindungsversuche (Reconnect) starten.

NF3 – Benutzbarkeit (Usability)

Das Bluetooth-Pairing mit dem Sensor darf maximal zwei manuelle Interaktionen erfordern.

Randbedingungen

R1 – Plattform-Kompatibilität

Die mobile Applikation muss nativ oder als hybride App auf Android-Geräten lauffähig sein.

3. Architekturentscheidungen (ADR / Abwägungen)

In diesem Abschnitt werden die zentralen Architekturentscheidungen (Architecture Decision Records) und Abwägungen bezüglich Hardware, App-Framework und Auswertungstechnologie dokumentiert.

Abwägungen

Hardware / Gerät

Kriterium	Vorgefertigtes Gerät (z. B. Fitbit)	Hybrid (Seeed XIAO nRF52840)	Eigene Lösung (Custom PCB)
Vorteile (+)	• Klein und ausgereift	• Kostengünstig • Integrierte IMU & BLE	• Maximale Kontrolle über Formfaktor und Sensoren
Nachteile (-)	• Teuer • Schlecht erweiterbar durch andere Sensoren	• Gehäuse muss selbst konstruiert/gedruckt werden • Höherer Integrationsaufwand als fertiges Konsumentenprodukt	• Sehr hohe Entwicklungskosten und Time-to-Market • Komplexes PCB-Design und Fertigungsrisiko

• *Entscheidung:** XIAO seed NRF52840

App-Entwicklung

Kriterium	Cross-Platform (Ionic / Flutter)	Hybrid (React Native)	Native (iOS Swift / Android Java)
Vorteile (+)	• Schnelles Prototyping und einfache Web-Technologien	• Hohe Code-Wiederverwendbarkeit • Schnelle UI-Iterationen	• Optimale Bluetooth-Leistung • Direkter API-Zugriff
Nachteile (-)	• Performance-Einbußen bei 50Hz Echtzeit-Diagrammen • Komplexere native Bluetooth-Anbindung über Plugins	• BLE-Bibliotheken von Drittanbietern erfordern Wartung • Performance-Overhead durch JS-Bridge bei kontinuierlichem Datenstrom	• Hohe Entwicklungskosten durch zwei separate Codebases • Längere Time-to-Market und aufwendige doppelte Pflege

• *Entscheidung:** Hybrid (React Native)

Bewegungsauswertung

Kriterium	Auf dem Gerät (Edge AI)	Komplett in der App	Hybrid
Vorteile (+)	• Per Edge Impulse einfach umzusetzende Inferenz	• Genug Rechenleistung für komplexe Deep-Learning-Modelle	• MCU filtert/komprimiert Daten • App übernimmt Inferenz
Nachteile (-)	• Begrenzte Speicher- und Rechenkapazitäten der MCU • Modell-Updates erfordern Firmware-Flashen	• Hohe BLE-Datenrate (50Hz Rohdatenstrom) erforderlich • Erhöhter Akkuverbrauch auf dem Mobilgerät	• Höhere Systemkomplexität durch geteilte Logik • Komplexeres Debugging bei Übertragungsverzögerungen

• *Entscheidung:** Auf dem Gerät (Edge-Inferenz)

• *Begründung:** Es gibt genügend Ressourcen, um ein kleines ML-Modell auf dem Mikrocontroller zu deployen und dafür zu trainieren.

Datenspeicherung

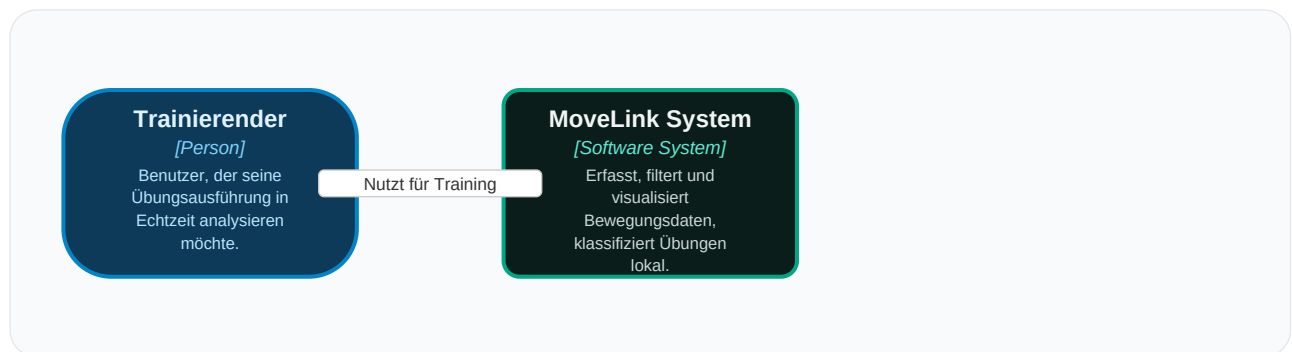
Kriterium	Lokal	Datenbank	Hybrid
Vorteile (+)	• Einfach und schnell umzusetzen • Keine komplexen Abhängigkeiten	• Skaliert gut	• MCU filtert/komprimiert Daten; App übernimmt Inferenz
Nachteile (-)	• Kann schlecht skalieren • Kein Online-Dienst	• Höhere Systemkomplexität durch geteilte Logik • Erhöhter Akkuverbrauch auf dem Mobilgerät	• Komplexeres Debugging bei Übertragungsverzögerungen

4. System-Architektur (C4 Modell)

Die Software-Architektur von MoveLink ist nach dem C4-Modell strukturiert. Im Folgenden werden die einzelnen Container des Systems (Embedded Firmware und Mobile App) im Detail beschrieben. Die Datenhaltung erfolgt lokal in der Mobile App. Für jeden dieser Unterpunkte werden die C4-Komponenten und Datenflüsse visuell und textuell dargestellt, gefolgt von den Anforderungen und der Architekturentscheidung (ADR).

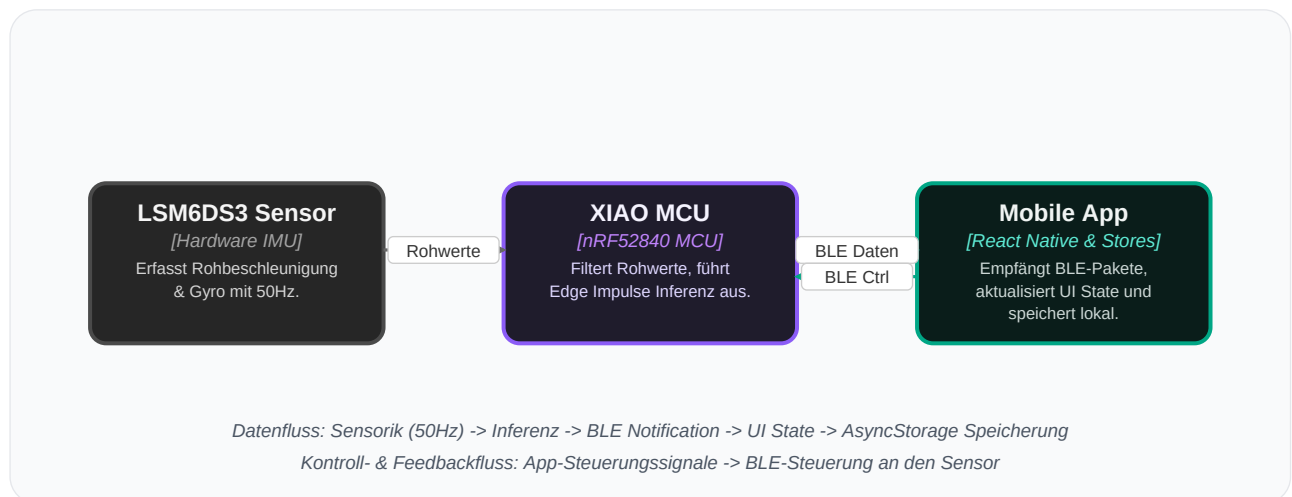
System-Kontext-Diagramm (C4 Level 1)

Das folgende System-Kontext-Diagramm zeigt die Position des MoveLink-Systems in seiner Betriebsumgebung und die Interaktion mit dem Trainierenden:



System-Daten- & Kontrollfluss-Diagramm

Das folgende Diagramm visualisiert den End-to-End Daten- und Kontrollfluss im Gesamtsystem:

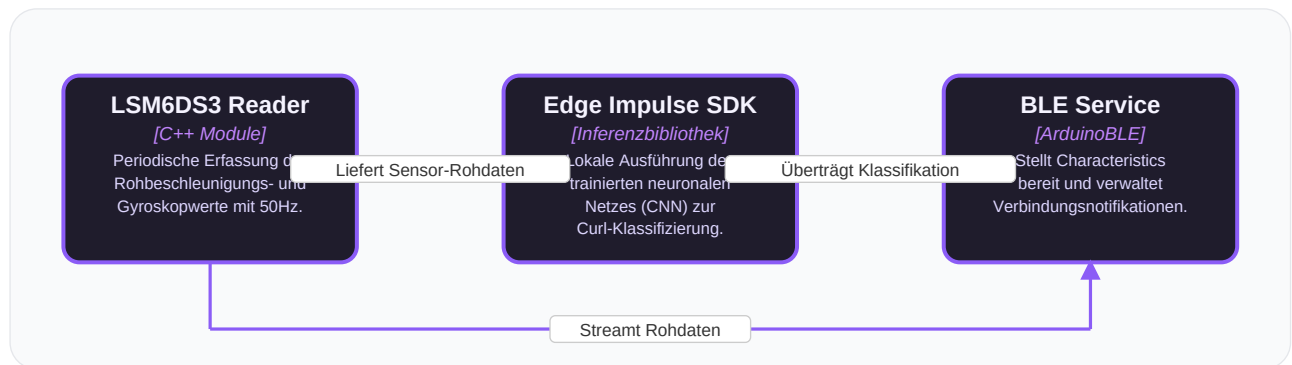


4.1 Embedded Sensor-Firmware (Xiao MCU)

Die Sensor-Firmware läuft auf dem **XIAO nRF52840 Sense Controller**. Sie erfasst Beschleunigungs- und Rotationsdaten über den LSM6DS3-Sensor, wendet Filter an und überträgt Daten per BLE oder wertet sie per Edge-Inferenz direkt auf dem Chip aus.

Architektur-Abbildung (Komponenten)

Das folgende Komponenten-Diagramm zeigt die inneren Module der Sensor-Firmware sowie die Kopplung von Sensor-Loop, Edge Impulse Inferenz und dem BLE Service:



4.1.1 Container-Architektur & Beschreibung

Dieses Dokument beschreibt die Embedded Sensor-Firmware als eigenständige, deploybare Einheit im C4-Modell.

C4-Architektur-Ebene

- **C4-Ebene:** Container
- **Deployable:** Ja
- **Deployment-Artefakt:** Binär-Firmware (flashed via USB/Serial)
- **Technologie-Stack:** Arduino C/C++, LSM6DS3 IMU Library, Edge Impulse SDK, Bluetooth Low Energy

Beschreibung

Die Sensor-Firmware läuft auf dem XIAO nRF52840 Sense Controller. Sie erfasst Beschleunigungs- und Rotationsdaten über den integrierten LSM6DS3-Sensor mit einer festen Abtastrate (50Hz), wendet Signalfilterungen zur Rauschunterdrückung an und streamt die Datenpakete als binäres Array via BLE Characteristics an die Mobile App. Alternativ führt sie Edge-Impulse-Inferenzmodelle direkt auf dem Mikrocontroller aus, um Trainingsübungen (z.B. Bizeps-Curls, Schulterdrücken und Seitheben) lokal zu klassifizieren und Fehler über die integrierten RGB-LEDs anzuzeigen.

Requirements

FA2.1 – Das Gerät sammelt die Dreh- und Beschleunigungsdaten des Trainierenden.

FA2.2 – Das Gerät erkennt, was für eine Bewegung ausgeführt worden ist.

FA2.3 – Das Gerät bewertet die Ausführung der Bewegung.

FA2.4 – Das Gerät gibt durch die LED den Verbindungsstatus aus.

FA2.5 – Das Gerät versendet die Daten an die App.

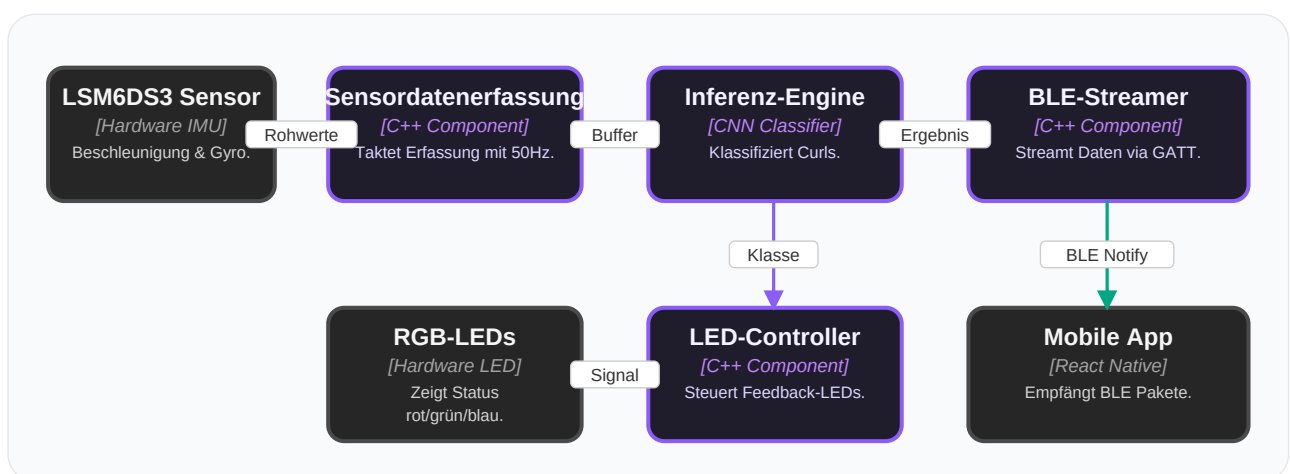
FA2.6 – Das Gerät ist in einem Gehäuse.

Komponenten in diesem Container

Die Sensor-Firmware besteht aus folgenden logischen Komponenten:

- **FA2.1** -> [Sensordatenerfassung (Loop)](file:///home/arch/repo/movelink/embedded/src/components/sensordatenerfassung/architecture.md)*: Liest kontinuierlich Beschleunigung (X, Y, Z) und Gyroskop (X, Y, Z).
- **FA2.2, FA2.3** -> [Inferenz-Engine (Edge Impulse)](file:///home/arch/repo/movelink/embedded/src/components/inferenz_engine/architecture.md)*: Klassifiziert Übungsausführungen lokal auf dem Chip.
- **FA2.4** -> [LED- & Display-Controller](file:///home/arch/repo/movelink/embedded/src/components/led_display_controller/architecture.md)*: Bietet direktes visuelles Feedback an den Nutzer bei Fehlern über RGB-LEDs.
- **FA2.5** -> [BLE-Streamer](file:///home/arch/repo/movelink/embedded/src/components/ble_streamer/architecture.md)*: Überträgt die erfassten Daten an den App-Container.
- **FA2.6** -> [Gehäuse](file:///home/arch/repo/movelink/embedded/src/components/gehause/architecture.md)*: Bietet physischen Schutz, sodass das Tragen erleichtert wird.

Datenfluss



4.1.7 Abwägungen & Architekturentscheidungen

Kriterium	Arduino	mbOS (Mbed OS)	Zephyr
Vorteile	• Extrem anfängerfreundlich und einfach zu bedienen
 • Riesige Community und unzählige fertige Bibliotheken
 • Sehr schnelles Prototyping ("Plug & Play")
 • Minimaler Konfigurationsaufwand	• Integriertes RTOS (Real-Time Operating System)
 • Starker Fokus auf IoT und Netzwerkverbindungen
 • Offizielle Unterstützung und Optimierung durch ARM	• Sehr mächtiges, echtes und hoch skalierbares RTOS
 • Breite Hardware-Unterstützung durch Device Trees
 • Hohe Modularität für professionelle, industrielle Projekte
Nachteile	• Kein natives RTOS (standardmäßig kein echtes Multithreading)
 • Bei sehr komplexen Softwarearchitekturen schwer skalierbar
 • Eingeschränkte harte Echtzeitfähigkeit	• Deutlich steilere Lernkurve als bei Arduino
 • Größerer Speicherbedarf (Overhead) für simple Aufgaben
 • Weniger Community-Ressourcen bei Fehlersuchen	• Sehr komplexes Build-System (West / CMake)
 • Hoher initialer Einarbeitungs- und Einrichtungsaufwand
 • Absoluter "Overkill" für einfache Steuerungsaufgaben

Architektur-Entscheidung

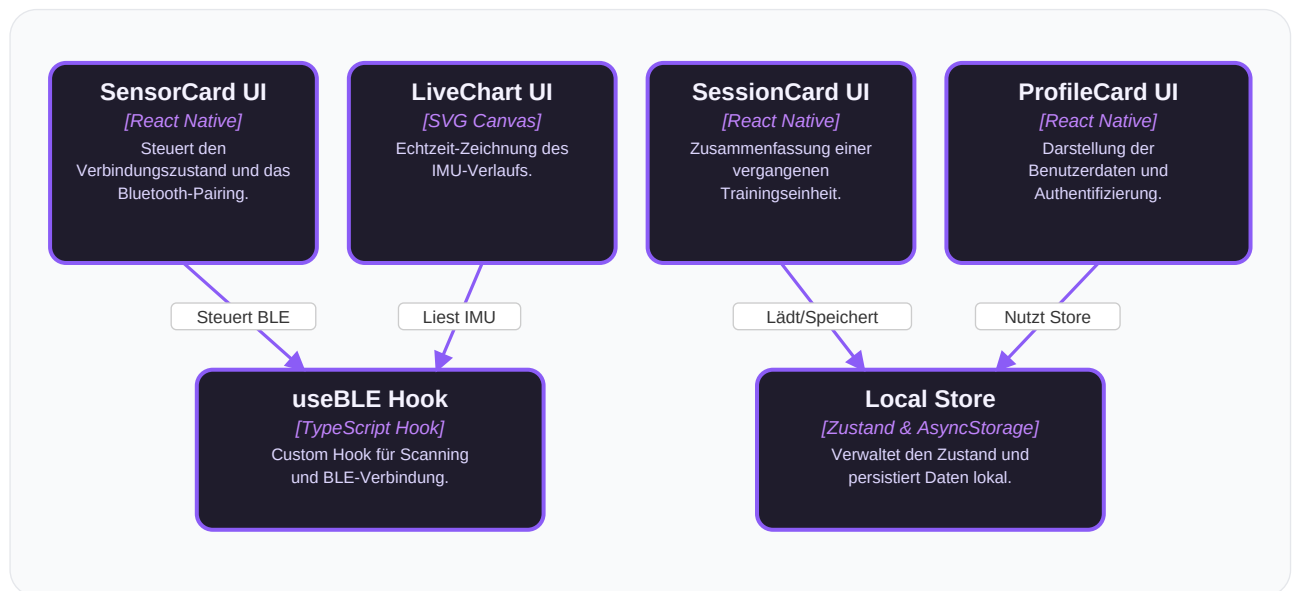
- **Entscheidung:** Arduino
- **Begründung:** Einfachheit. Der Mehraufwand, der durch die Einarbeitung und Konfiguration komplexerer RTOS-Systeme wie mbOS oder Zephyr entstehen würde, bringt für die Anforderungen dieses Projekts keine nennenswerten Vorteile. Arduino bietet durch seine unkomplizierte Handhabung und die breite Bibliotheksunterstützung den effizientesten Weg zur Umsetzung.

4.2 Mobile App (React Native)

Die Mobile App bildet das Benutzer-Interface (UI) des Systems. Sie verbindet sich per BLE mit der Xiao MCU, visualisiert Live-Bewegungsdaten und synchronisiert sie mit dem Backend.

Architektur-Abbildung (Komponenten)

Das folgende Komponenten-Diagramm veranschaulicht die Benutzeroberflächen-Module und Custom React Hooks des App-Containers:



<!--

C4-Ebene: Container

Deployable: Ja

• ->

Dieses Dokument beschreibt die Mobile App als eigenständige, deploybare Einheit im C4-Modell.

C4-Architektur-Ebene

- **C4-Ebene:** Container
- **Deployable:** Ja
- **Deployment-Artefakt:** Android Package (.apk) / iOS IPA
- **Technologie-Stack:** React Native, Expo, TypeScript, Zustand, BLE PLX, AsyncStorage

Beschreibung

Die MoveLink Mobile App ist die primäre Benutzerschnittstelle des Systems. Sie läuft auf Android- und iOS-Endgeräten und verbindet sich über Bluetooth Low Energy (BLE) mit dem embedded Sensor-Gerät, um Bewegungsdaten in Echtzeit zu erfassen, zu visualisieren und zur persistenten Speicherung lokal auf dem Gerät zu speichern.

Requirements

FA1.1 – Die App bietet eine Navigationsstruktur (Reiter/Menü) für Hardwaregeräte, Trainings und vergangene Einheiten.

FA1.2 – Die App ermöglicht das Scannen nach verfügbaren Bluetooth-Sensorgeräten.

FA1.3 – Die App baut eine stabile Bluetooth-Verbindung zum Sensor auf.

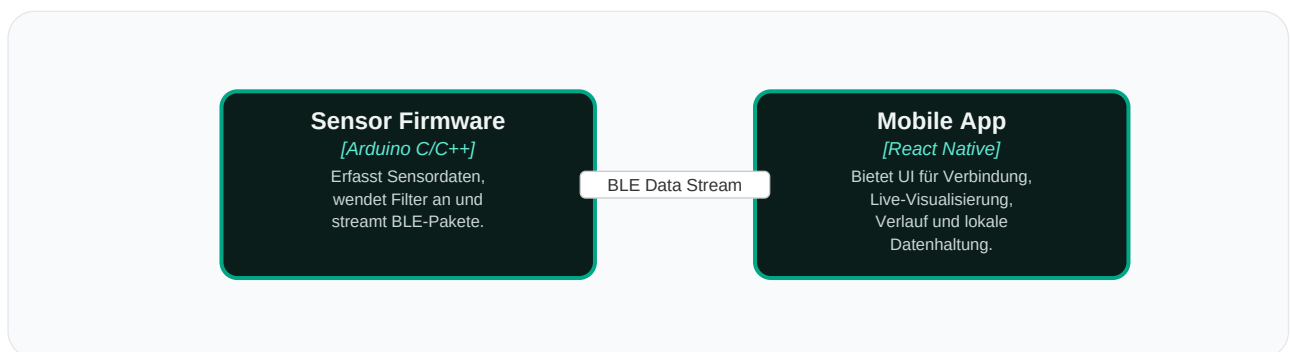
FA1.4 – Die App demonstriert grafisch die auszuführende Übungsausführung (z. B. via Lottie-Animation).

FA1.5 – Die App visualisiert die empfangenen Sensordaten und den Übungsfortschritt in Echtzeit.

FA1.6 – Die App zeigt historische Trainingseinheiten grafisch und statistisch an.

FA1.7 – Das System speichert aufgezeichnete Trainingseinheiten (Übungstyp, Wiederholungen, Qualität) persistent zur historischen Auswertung.

Datenfluss



Komponenten in diesem Container

Die App enthält mehrere Komponenten (C4-Komponenten-Ebene):

1. **[SideNav](file:///c:/Users/erlin/repo/movelink/app/components/side_nav/architecture.md):**
Navigationskomponente für die App-Steuerung. (Erfüllt: **FA1.1**, **FA1.1.1**)
2. **[SensorCard](file:///c:/Users/erlin/repo/movelink/app/components/sensor_card/architecture.md):**
Verwaltung der BLE-Geräteverbindung und Pairing. (Erfüllt: **FA1.2**, **FA1.3**, **NF3**)
3. **[LiveChart](file:///c:/Users/erlin/repo/movelink/app/components/live_chart/architecture.md):**
Echtzeit-Visualisierung der IMU-Beschleunigungs- und Gyroskopwerte. (Erfüllt: **FA1.5**)
4. **[SessionCard](file:///c:/Users/erlin/repo/movelink/app/components/session_card/architecture.md):**
Visualisierung historischer Trainingseinheiten. (Erfüllt: **FA1.6**)
5. **[BLE-Hook (useBLE)](file:///c:/Users/erlin/repo/movelink/app/hooks/useBLE.ts):** Kapselt die Bluetooth-Gerätekommunikation und den Reconnect. (Erfüllt: **FA1.3**, **NF2**)
6. **[ProfileCard](file:///c:/Users/erlin/repo/movelink/app/components/ProfileCard/architecture.md):**
Visualisierung der Benutzerprofildetails. (Erfüllt: **FA1.7**)
7. **[Local Store](file:///c:/Users/erlin/repo/movelink/app/store/index.ts):** Verwaltet den App-Zustand und persistiert die Trainingsdaten lokal über AsyncStorage. (Erfüllt: **FA1.7**, **FA3**)

Architekturentscheidung (ADR / Abwägungen)

Entscheidung: Verwendung von **React Native (Hybrid-App)** zur plattformübergreifenden Entwicklung mit hoher Code-Wiederverwendbarkeit und schnellen UI-Iterationen, bei gleichzeitiger Optimierung des SVG-Diagramm-Renderings für den 50Hz Datenstrom.

5. Rückverfolgbarkeits-Matrix (Traceability)

Die folgende Matrix veranschaulicht die Beziehungen und Verknüpfungen zwischen den Use Cases, den funktionalen Anforderungen (FA), den Systemkomponenten und den entsprechenden Klassen/Code-Dateien im System. Klicken Sie auf eine Anforderungs-ID, um zu deren Definition zu springen, oder auf eine Komponente, um deren C4-Modell-Kapitel aufzurufen.

| Use Case | Map | FA Ebene 1 | Sub-FA | Component | Detail-FA Ebene 3 | Klassenebene |
|----------|-----|------------|--------|-----------|-------------------|--------------|
|----------|-----|------------|--------|-----------|-------------------|--------------|

| | | | | | | |
|---|--|--|-------|--------------------------------|---|---|
| UC-1
Trainingsgerät
verbinden | | FA1
Applikation (Mobile App) | FA1.1 | SideNav | FA1.1.1 Die App bietet eine Seiten-Navigation (Drawer/SideNav) auf Tablets und Web. | (Zielstruktur) |
| | | | FA1.2 | SensorCard | | Props </> StatusLabel() </> ScanningDots() </> SensorCard() </> |
| | | | FA1.3 | SensorCard | - | isConnected() </> StatusLabel() </> ScanningDots() </> style SensorCard() </> |
| | | | FA1.4 | - | - | isConnected() </> isActive </> color </> styles |
| | | | FA1.5 | LiveChart | FA1.5.1 Die App visualisiert die empfangenen Sensordaten und den Anomalie-Score | Props </> HEIGHT </> buildPaths() </> allVals </> min </> max </> path </> last </> max </> |
| UC-2
Echtzeit-Training
überwachen | | FA2
Trainingsgerät
(Sensor-Firmware) | FA1.6 | SessionCard | FA1.6.1 Die App zeigt was für Übungen ausgeführt wurden: | Props </> HEIGHT </> buildPaths() </> allVals </> min </> max </> path </> last </> max </> |
| | | | | | - | Props </> HEIGHT </> buildPaths() </> allVals </> min </> max </> path </> last </> max </> |
| | | | FA2.1 | Sensordatenerfassung (Loop) | FA2.1.1 Beschleunigung wird eingelesen | Props </> HEIGHT </> buildPaths() </> allVals </> min </> max </> path </> last </> max </> |
| | | | | | FA2.1.2 Drehung wird eingelesen | Props </> HEIGHT </> buildPaths() </> allVals </> min </> max </> path </> last </> max </> |
| UC-3
Trainingsdaten
einsehen | | FA2
Trainingsgerät
(Sensor-Firmware) | FA2.2 | Inferenz-Engine (Edge Impulse) | FA2.2.1 Das Gerät erkennt einen Idle Modus | (Zielstruktur) |
| | | | | | FA2.2.2 Das Gerät erkennt einen Curl | (Zielstruktur) |
| | | | | | FA2.2.3 Das Gerät erkennt einen shoulder press | (Zielstruktur) |
| | | | | | FA2.2.4 Das Gerät erkennt einen Lateral raise | (Zielstruktur) |
| | | | | | FA2.2.5 Das Gerät erkennt eine tricep extension | (Zielstruktur) |
| | | | FA2.3 | Inferenz-Engine (Edge Impulse) | - | (Zielstruktur) |
| | | | FA2.4 | LED- & Display-Controller | FA2.4.1 Ansteuerung der RGB-LED zur Signalisierung von Fehlern (Rot), perfekten | (Zielstruktur) |
| | | | FA2.5 | BLE-Streamer | FA2.5.1 Ansteuerung der Bluetooth LE zur Signalisierung und Übertragung der Sensordaten mit minimalem | (Zielstruktur) |
| | | | FA2.6 | Gehäuse | FA2.6.1 Schutz der Elektronik vor äußeren Einflüssen. | (Zielstruktur) |
| | | | | | FA2.6.2 Befestigungsmöglichkeit (z.B. Sportarmband) für stabile Fixierung während des Trainings. | (Zielstruktur) |